

Understanding Attention

From Q, K, V to Modern Transformer Attention

A technical handbook for understanding what attention computes, why the equation has its particular form, and how modern LLM systems execute it.

What this handbook promises

By the end of this handbook, you should be able to look at

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

and explain every object and operation in it: where Q , K , and V come from; what each element of $QK^\top$ means; why the scores are scaled; how masks change the allowed connections; why softmax is applied row-wise; and why multiplying by V produces a new representation for every query position.

You should also be able to explain how the same core mechanism appears in self-attention, cross-attention, causal language models, multi-head attention, KV-cached decoding, MQA, GQA, MLA, FlashAttention, and sparse/local attention.

Prerequisites

You need basic comfort with vectors, matrices, matrix multiplication, dot products, softmax, and neural-network linear layers. No prior knowledge of Transformer internals is assumed.

Scope

This is a handbook about *attention*. It touches RoPE, KV caching, MQA/GQA/MLA, and FlashAttention only to explain how they modify, support, or execute attention. Full inference serving, PagedAttention, long-context training, and model parallelism deserve separate handbooks.

Accuracy rule

Named methods are grounded in their primary papers; foundational explanations are cross-checked against current Stanford course material; implementation behavior is checked against current PyTorch documentation. Claims about historical papers are kept within what those papers actually establish. [1, 2, 3, 15, 16, 17]

Contents

What this handbook promises	2
1 Before the Transformer: why attention appeared	4
2 Attention as content-dependent weighted aggregation	5
3 Notation and tensor shapes	6
4 Where Q, K, and V come from	7
5 The score matrix: $QK^\top$	8
6 Why divide by $\sqrt{d_k}$?	9
7 Masks: which connections are allowed?	10
8 Softmax and value aggregation	11
9 Worked example I: Q, K, V and raw scores	12
10 Worked example II: softmax weights	13
11 Worked example III: the output vectors	14
12 Self-attention vs cross-attention	15
13 Multi-head attention	16
14 Position: attention needs an ordering signal	17
15 Causal attention in an autoregressive language model	18
16 Training vs decoding	19
17 The KV cache	20
18 MHA, MQA, and GQA	21
19 MLA: compressing the KV representation	22
20 What exactly is quadratic?	23
21 FlashAttention: same attention, different execution	24
22 FlashAttention generations: optimize the bottleneck you actually have	25
23 Sparse and local attention	26
24 A transparent PyTorch implementation	27
25 Production SDPA in PyTorch	28
26 Common implementation mistakes	29
27 Common misconceptions	30
28 Are attention weights explanations?	31
29 The complete mental model	32
References and further reading	33

1 Before the Transformer: why attention appeared

Early neural sequence-to-sequence systems often encoded an entire source sequence into a single fixed-length vector and asked the decoder to generate the target sequence from that representation. Bahdanau, Cho, and Bengio argued that this fixed-vector interface could become a bottleneck, especially as source sequences grew longer. Their model allowed the decoder to form a different context vector at each output step by softly weighting encoder states according to their relevance to the current decoding state. [1]

The important idea was not yet “self-attention.” The decoder was using a learned compatibility mechanism to decide which source representations should contribute more strongly to the current output. Luong, Pham, and Manning later studied multiple NMT attention variants, including global attention over all source positions and local attention over a subset. [2]

The 2017 Transformer changed the role of attention much more radically. Instead of using recurrence or convolution as the main mechanism for moving information through a sequence, it built its encoder and decoder around attention plus position-wise feed-forward layers. The paper describes self-attention as relating different positions of a single sequence to compute new representations. [3]

Mental model. Attention did not begin as “the Transformer equation.” The durable idea is learned, content-dependent aggregation: a query determines how strongly different key-value pairs contribute to an output. The Transformer made a particular attention formulation the central sequence-mixing operation.

What we will study

The rest of the handbook focuses on **scaled dot-product attention**, because that is the form used by the Transformer and by modern LLM attention kernels. Earlier additive-attention mechanisms are historically important, but their scoring function differs.

2 Attention as content-dependent weighted aggregation

An attention function can be described as mapping a **query** and a set of **key-value pairs** to an output. Each key is used to determine how compatible its corresponding value is with the query; the output is a weighted combination of the values. This general description appears directly in the Transformer paper. [3]

For one query vector q and key-value pairs (k_j, v_j) , scaled dot-product attention can be written

$$s_j = \frac{q^\top k_j}{\sqrt{d_k}}, \quad a_j = \frac{e^{s_j}}{\sum_r e^{s_r}}, \quad o = \sum_j a_j v_j.$$

The score s_j is a learned compatibility signal because q and k_j themselves come from learned projections. Softmax turns the scores into nonnegative weights that sum to one across the allowed keys. The output o is then a weighted average in value space.

This language is more precise than common personifications such as “the query asks a question and the key contains the answer.” Those analogies can be memorable, but they are not the mechanism. A query and key are vectors in learned representation spaces. Their dot product supplies a scalar compatibility score.

Mental model. The key chooses the *weight*; the value supplies the *content being mixed*. That separation is the conceptual reason attention uses both K and V .

A useful abstraction

For many queries at once, attention becomes matrix computation:

$$S = QK^\top, \quad A = \text{softmax}(S/\sqrt{d_k}), \quad O = AV.$$

The next pages unpack each object and its shape.

3 Notation and tensor shapes

Let

- B = batch size,
- L = number of query positions,
- S = number of key/value positions,
- H_q = number of query heads,
- H_{kv} = number of key/value heads,
- d_h = per-head query/key dimension,
- d_v = per-head value dimension.

For a single head and a single example, we can ignore batch/head dimensions and write

$$Q \in \mathbb{R}^{L \times d_h}, \quad K \in \mathbb{R}^{S \times d_h}, \quad V \in \mathbb{R}^{S \times d_v}.$$

Then

$$QK^\top \in \mathbb{R}^{L \times S}, \quad A \in \mathbb{R}^{L \times S}, \quad AV \in \mathbb{R}^{L \times d_v}.$$

Two details matter.

First, the attention matrix is not always square. In self-attention over a single full sequence, $L = S$. In cross-attention, or in cached decoding where a short query block attends to a longer prefix, L and S can differ. Current PyTorch SDPA documentation explicitly uses separate query length L and source length S . [17]

Second, modern grouped attention can have $H_q \neq H_{kv}$. Standard MHA usually has the same number of query and KV heads; MQA uses one KV head; GQA uses an intermediate number. [4, 9]

Important. Do not silently use $n \times n$ as the universal shape of attention weights. It is correct for same-length self-attention, but it obscures cross-attention and incremental inference.

4 Where Q, K, and V come from

Suppose the input to an attention layer is a matrix of token representations

$$X \in \mathbb{R}^{n \times d_{\text{model}}}.$$

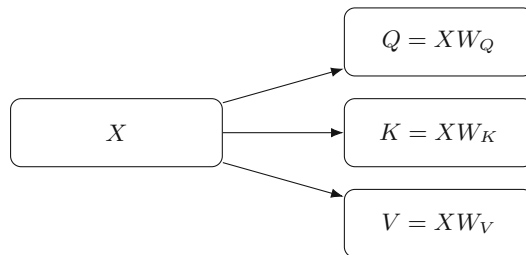
A standard self-attention layer applies learned linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

For one head,

$$W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_h}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

In multi-head attention, each head conceptually has different learned projections, or an implementation computes one larger projection and reshapes it into heads. The Transformer paper defines multi-head attention in terms of separate learned projections for each head. [3]



Q , K , and V are therefore not fixed semantic roles attached to tokens. They are *learned views* of the current hidden representations. A token can have different query/key/value vectors in different layers and different heads.

Mental model. The model learns two spaces for deciding compatibility (Q and K) and a space for the information that will be aggregated (V).

5 The score matrix: $QK^\top$

Take query row q_i and key row k_j . Their dot product is

$$S_{ij} = q_i^\top k_j.$$

The complete matrix multiplication $QK^\top$ computes every query-key dot product in parallel:

$$QK^\top = \begin{bmatrix} q_1^\top k_1 & q_1^\top k_2 & \cdots & q_1^\top k_S \\ q_2^\top k_1 & q_2^\top k_2 & \cdots & q_2^\top k_S \\ \vdots & \vdots & \ddots & \vdots \\ q_L^\top k_1 & q_L^\top k_2 & \cdots & q_L^\top k_S \end{bmatrix}.$$

Every row corresponds to one query position. Every column corresponds to one key position.

The dot product is not a probability. It is an unnormalized compatibility score. It can be positive or negative and its magnitude depends on the scale and dimension of the vectors.

	k_1	k_2	k_3	k_4	$\dots$	k_S	
q_1	$q_1^\top k_1$						
q_2			$q_2^\top k_3$				
q_3							
q_4							

$L \times S$ scores

Important. Calling the score “similarity” is convenient but incomplete. Dot-product attention learns projections specifically so the resulting compatibility is useful for the task. It need not behave like a generic semantic-similarity metric.

6 Why divide by $\sqrt{d_k}$?

The Transformer scales the query-key dot products before softmax:

$$\tilde{S} = \frac{QK^\top}{\sqrt{d_k}}.$$

The paper gives a specific motivation. Suppose the components of q and k are independent random variables with mean zero and variance one. Then

$$q^\top k = \sum_{r=1}^{d_k} q_r k_r$$

has mean zero and variance d_k . As d_k grows, unscaled dot products can therefore grow in magnitude. Large-magnitude logits can push softmax into a saturated regime with very small gradients. Dividing by $\sqrt{d_k}$ normalizes this illustrative variance from d_k to approximately one. [3]

A short derivation under those assumptions is

$$\text{Var}(q^\top k) = \sum_{r=1}^{d_k} \text{Var}(q_r k_r) = d_k,$$

so

$$\text{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) \approx 1.$$

Important. This derivation is a motivation, not a claim that learned query/key components in a trained Transformer are always independent, zero-mean, unit-variance random variables. The paper explicitly presents those assumptions as an illustration of why dot products can grow with dimension.

Mental model. Scaling is about keeping score magnitudes in a range where softmax behaves usefully as dimensionality grows. It is not a normalization of Q or K into unit vectors.

7 Masks: which connections are allowed?

Before softmax, an attention implementation can add a mask or bias to the score matrix. For causal language modeling, future key positions must not influence an earlier query position. A common additive mask is

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Then

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right).$$

Because $e^{-\infty} = 0$, masked entries receive zero probability after softmax.

For a length-4 square causal attention matrix, the allowed pattern is

$$\begin{bmatrix} \checkmark & \times & \times & \times \\ \checkmark & \checkmark & \times & \times \\ \checkmark & \checkmark & \checkmark & \times \\ \checkmark & \checkmark & \checkmark & \checkmark \end{bmatrix}.$$

The original Transformer decoder implements autoregressive masking by setting illegal pre-softmax connections to $-\infty$. [3]

Masks are broader than causality. Padding masks can prevent attention to padding tokens; structured attention can restrict connectivity to local windows, blocks, or other patterns. Current PyTorch SDPA accepts Boolean or additive floating-point masks and separately supports a causal mode. [17]

Important. In non-square attention ($L \neq S$), “lower triangular” requires an alignment convention. Current PyTorch documentation explicitly distinguishes square and non-square causal bias behavior. Do not blindly construct an $L \times S$ triangle without checking what the API expects.

8 Softmax and value aggregation

After scaling and masking, softmax is applied across the key dimension for each query row:

$$A_{ij} = \frac{\exp(\tilde{S}_{ij})}{\sum_{r=1}^S \exp(\tilde{S}_{ir})}.$$

Therefore, for each query i ,

$$A_{ij} \geq 0, \quad \sum_{j=1}^S A_{ij} = 1.$$

The output at query position i is

$$o_i = \sum_{j=1}^S A_{ij} v_j.$$

In matrix form,

$$O = AV.$$

This final multiplication is easy to overlook, but it is where attention actually *moves information*. The score matrix only decides how much each value contributes. The values are the vectors that are mixed into the new representation.

If a particular query gives weights $(0.7, 0.2, 0.1)$ to three values, its output is

$$0.7v_1 + 0.2v_2 + 0.1v_3.$$

The output remains in the value-vector space, not in score space.

Mental model. Scores answer “how much?” Values answer “what gets aggregated?” The complete attention operation is a content-dependent mixing operator.

Why row-wise?

Each query independently produces a distribution over its permitted key positions. Column-wise softmax would normalize across queries instead and would describe a different operation.

9 Worked example I: Q, K, V and raw scores

We now compute one causal attention head completely by hand. Use

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}, \quad d_k = 2.$$

There are three query positions and three key/value positions. The query-key dot products are

$$QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

After dividing by $\sqrt{2}$,

$$\frac{QK^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

Now impose a causal mask. Query 1 may use only key 1. Query 2 may use keys 1–2. Query 3 may use all three keys:

$$\tilde{S}_{\text{causal}} \approx \begin{bmatrix} 0.7071 & -\infty & -\infty \\ 0 & 0.7071 & -\infty \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

All numerical values in this worked example were independently recomputed programmatically before layout.

10 Worked example II: softmax weights

Apply softmax row by row.

For the first query, only one position is legal, so

$$A_1 = [1, 0, 0].$$

For the second query, the legal logits are $[0, 0.7071]$:

$$A_2 \approx [0.3302, 0.6698, 0].$$

For the third query, the logits are $[0.7071, 0.7071, 1.4142]$:

$$A_3 \approx [0.2483, 0.2483, 0.5035].$$

Therefore

$$A \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.3302 & 0.6698 & 0 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}.$$

Every row sums to one (up to rounding), and all causally forbidden positions have weight zero.

Notice what the third row says: the third query assigns about half its attention weight to the third value and about one quarter to each earlier value. These numbers are not annotations added by us; they are the normalized result of the learned-compatibility calculation in this toy setup.

Important. In a trained model, attention weights are produced inside many heads and layers and interact with residual connections, normalization, feed-forward networks, and later computation. A high attention weight should not automatically be interpreted as a complete explanation of the model's prediction. We return to this later.

11 Worked example III: the output vectors

Finally multiply the attention weights by the value matrix:

$$O = AV.$$

Using

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix},$$

we obtain

$$O \approx \begin{bmatrix} 1 & 0 \\ 0.3302 & 1.3395 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

The second row, for example, is

$$0.3302[1, 0] + 0.6698[0, 2] = [0.3302, 1.3396] \text{ (rounding)}.$$

The third row is

$$0.2483[1, 0] + 0.2483[0, 2] + 0.5035[3, 1] \approx [1.7587, 1.0000].$$

The entire attention mechanism is now visible:

$$Q, K \longrightarrow QK^\top \longrightarrow \frac{QK^\top}{\sqrt{d_k}} \longrightarrow \text{mask} \longrightarrow \text{softmax} \longrightarrow A \longrightarrow AV.$$

Mental model. If you can explain where every number in this three-token example comes from, the core attention equation is no longer a black box. Everything that follows modifies the source of Q/K/V, the permitted connectivity, the head structure, the positional treatment, or the way this computation is executed efficiently.

12 Self-attention vs cross-attention

The attention equation does not require Q, K, and V to come from the same sequence.

Self-attention

In self-attention, Q, K, and V are projections of representations from the same sequence (or same layer input):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

The Transformer encoder uses self-attention in this form. A causal decoder also uses self-attention, but masks future positions. [3]

Cross-attention

In encoder-decoder attention, queries come from the decoder side while keys and values come from encoder outputs:

$$Q = YW_Q, \quad K = XW_K, \quad V = XW_V.$$

If the target/query sequence has length L and the source has length S , the score matrix has shape $L \times S$.

	Q source	K,V source
Self-attention	same sequence	same sequence
Cross-attention	query/target stream	source/memory stream
Causal self-attention	same sequence	same sequence, masked

Important. “Self-attention” describes where Q/K/V come from. “Causal attention” describes a connectivity constraint. They are not synonyms: self-attention can be bidirectional or causal.

13 Multi-head attention

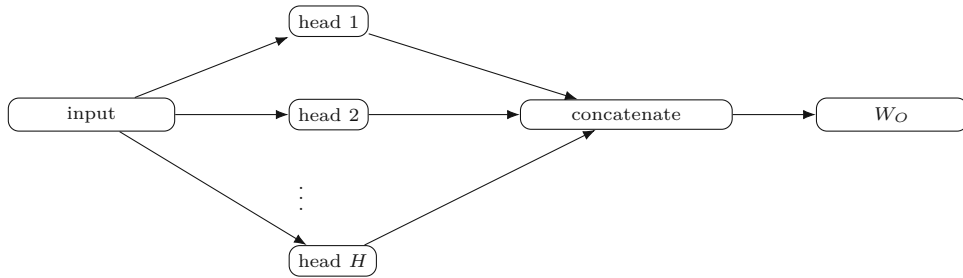
A single attention head produces one learned compatibility-and-aggregation pattern. Multi-head attention performs several attention operations in parallel using different learned projections, concatenates the resulting head outputs, and applies an output projection. The Transformer defines

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O,$$

where

$$\text{head}_h = \text{Attention}(QW_h^Q, KW_h^K, VW_h^V).$$

[3]



If d_{model} is held fixed and each head has dimension about d_{model}/H , increasing the number of heads does not simply multiply the full-dimensional attention cost by H ; each head is narrower. The original Transformer used this design to keep total cost comparable to a single full-dimensional head. [3]

Important. Do not assume each head corresponds to one clean human-interpretable concept. Different heads can learn different patterns, but head semantics are emergent and not guaranteed to map to simple labels.

14 Position: attention needs an ordering signal

Plain noncausal self-attention without any position-dependent signal is permutation-equivariant: if the input rows are permuted, the outputs are permuted in the same way. It has no inherent notion that one token is “third” or that another token is “five positions away.” The original Transformer therefore added positional encodings to token embeddings. [3]

Modern decoder-only LLMs often use Rotary Position Embedding (RoPE). RoPE rotates query and key components by position-dependent angles. In a simplified 2D block,

$$R_m(\theta) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}.$$

If $q_m = R_m q$ and $k_n = R_n k$, then

$$q_m^\top k_n = q^\top R_m^\top R_n k = q^\top R_{n-m} k,$$

so the attention interaction contains relative displacement information. RoFormer presents RoPE as encoding absolute position through rotations while introducing explicit relative-position dependence in self-attention. [7]

Two nuances matter.

First, positional encoding is not the same thing as attention. It modifies the representations or Q/K interaction so attention can use position.

Second, a causal mask itself introduces directional order constraints, but it does not by itself provide a rich representation of distance or absolute position.

Important. Being mathematically able to evaluate RoPE at positions beyond training length does not guarantee reliable long-context behavior. Context extrapolation depends on training and positional configuration, not on RoPE alone.

15 Causal attention in an autoregressive language model

An autoregressive language model factorizes a token sequence as

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

When training a decoder-only Transformer on a full sequence, the model can process many positions in parallel because a causal mask prevents representation at position t from using future tokens. The masked self-attention at each layer enforces the allowed dependency pattern. [3]

For query position t , attention can use keys/values from positions $1, \dots, t$, but not $t+1, \dots, T$.

This is one reason training and generation feel very different computationally. During training, the full known sequence is available and large matrix multiplications can process all query positions together. During generation, token $t+1$ does not exist until token t has been generated, so decoding is sequential across generated tokens.

Mental model. Causal masking makes parallel *training over positions* compatible with an autoregressive objective. It does not make autoregressive *generation* parallel, because future generated tokens are not yet known.

The attention calculation itself remains scaled dot-product attention. What changes during inference is how much prior computation can be reused.

16 Training vs decoding

Consider a causal decoder layer processing a prefix of length T .

Training / full-prefix processing

For a complete training sequence or prompt prefill, many query positions are processed together. Dense attention forms interactions across the relevant query-key pairs, typically through large batched matrix operations.

Incremental decoding

After the prefix has been processed, the model generates a new token. Its new hidden state produces a new query, key, and value at each layer. Crucially, the keys and values for earlier positions do not need to be recomputed from scratch: because of causal attention, earlier token representations were already determined without access to future tokens.

That observation leads to the KV cache.

	Prefill / training-like pass	Incremental decode
Queries processed	many positions	usually one/few new positions
Past K/V	computed in pass	reused from cache
Parallelism across sequence	high	generation remains sequential
Typical bottleneck emphasis	compute + memory	cache bandwidth/capacity + per-token latency

Important. The terms “prefill” and “decode” describe serving phases, not different mathematical definitions of attention. They expose different shapes and hardware bottlenecks for the same model.

17 The KV cache

In causal incremental decoding, each layer stores the key and value vectors produced for prior tokens. When a new token arrives, the layer computes the new token’s query/key/value, appends its K/V to the cache, and uses the new query to attend over cached keys and values.

For standard attention, an approximate cache-element count per token is

$$2 \times L_{\text{layers}} \times H_{kv} \times d_h,$$

where the factor 2 accounts for K and V. In bytes, multiply by storage bytes per element.

For illustration, with 32 layers, $d_h = 128$, and 2-byte K/V elements:

KV heads	cache / token	cache for 8192 tokens
32	512 KiB	4.0 GiB
8	128 KiB	1.0 GiB
1	16 KiB	0.125 GiB

This example isolates K/V storage and ignores allocator overhead, metadata, other activations, weights, and implementation-specific representations.

KV caching avoids repeatedly recomputing K/V for the entire prefix at every generated token. But it does not make dense attention independent of context length: a new query still interacts with the cached prefix, and cache memory grows with sequence length.

Important. “KV cache makes decoding $O(1)$ ” is false for standard dense attention. The new token’s attention work grows with the number of cached key/value positions, even though prior K/V projections are reused.

18 MHA, MQA, and GQA

The KV cache makes the number of key/value heads an inference-systems decision, not merely an architectural detail.

Multi-head attention (MHA)

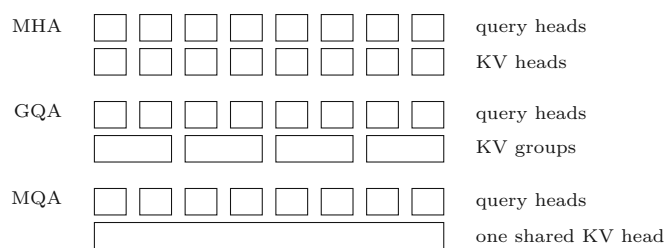
Standard MHA has separate K/V projections per attention head. If there are H query heads, there are typically H KV heads as well.

Multi-query attention (MQA)

Shazeer proposed MQA for faster incremental Transformer decoding. It keeps multiple query heads but shares a single key head and a single value head across them, greatly shrinking cached K/V tensors and memory-bandwidth demands. [4]

Grouped-query attention (GQA)

GQA generalizes MQA by using an intermediate number of KV heads: more than one, but fewer than the query heads. Query heads are divided into groups that share a KV head. Ainslie et al. introduced GQA as a way to approach MQA-like inference efficiency while retaining quality closer to MHA in their experiments. [9]



Mental model. MQA/GQA reduce KV heads, not query heads. Query-head multiplicity can remain high while the cached key/value representation is shared.

19 MLA: compressing the KV representation

Multi-head Latent Attention (MLA), introduced in DeepSeek-V2, approaches KV-cache reduction differently from MQA/GQA. Instead of primarily sharing K/V heads, MLA uses low-rank joint compression of key and value information. [11]

In the paper’s notation, a compressed latent is formed as

$$c_t^{KV} = W^{DKV} h_t,$$

then key/value content can be derived through learned up-projections. The core motivation is that the cached state can be much smaller than storing conventional per-head keys and values.

However, a precise description needs one important caveat: DeepSeek-V2 also uses **decoupled RoPE**. Position-sensitive key components associated with RoPE are handled separately because directly applying RoPE inside the compressed-key path would prevent some projection absorption used for efficient inference. The paper therefore reports a cache containing both the compressed latent and a decoupled positional key component, not literally only one bare latent vector in the complete implementation. [11]

Mechanism	Main KV-cache idea	KV heads
MHA	store per-head K and V	many
MQA	share one K/V head	one
GQA	share K/V within groups	intermediate
MLA	low-rank joint KV compression (+ positional component)	different formulation

Important. MLA is not just “GQA with fewer heads.” It changes the representation being cached and the projection algebra used during inference.

20 What exactly is quadratic?

The phrase “attention is quadratic” compresses several different resource questions.

For dense self-attention with sequence length n and representation/head dimensions held fixed, the original Transformer analysis gives a per-layer self-attention term $O(n^2d)$. [3]

But three quantities must be separated:

1. **Arithmetic work.** Computing all dense query-key interactions and applying them to values has a quadratic sequence-length term.
2. **Materialized intermediate memory.** A naive implementation may explicitly store the $n \times n$ score/probability matrix, giving quadratic intermediate storage.
3. **Memory traffic.** Reading/writing those intermediates between GPU memory levels can dominate runtime even if the arithmetic count is unchanged.

These distinctions explain why two statements can both be true:

- Dense attention still performs quadratic pairwise interactions.
- An exact attention implementation can avoid materializing the entire quadratic attention matrix in high-bandwidth memory and run much faster.

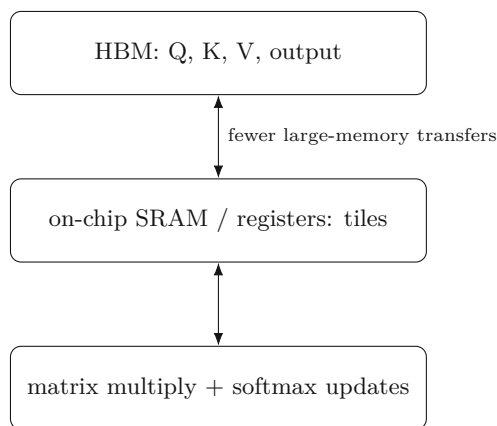
That second statement is the core of FlashAttention.

Mental model. Complexity, memory footprint, memory traffic, and wall-clock time are related but not interchangeable. Never use one as a synonym for another.

21 FlashAttention: same attention, different execution

FlashAttention is an IO-aware **exact** attention algorithm. Its key insight is to account explicitly for the GPU memory hierarchy: transfers between high-bandwidth memory (HBM) and faster on-chip SRAM can be a dominant cost. Instead of materializing the full attention matrix in HBM, FlashAttention tiles the computation so blocks of Q/K/V and intermediate statistics can be processed on-chip, reducing HBM reads/writes. [8]

The mathematical target remains scaled dot-product attention. The algorithm uses an online/blocked softmax strategy so exact results can be assembled without storing all scores at once in HBM.



The original paper emphasizes fewer HBM accesses, not a replacement of dense attention with an approximation. Stanford CS336 uses FlashAttention as a systems exercise precisely to connect GPU memory hierarchy with attention performance. [8, 16]

Important. FlashAttention does not turn standard dense attention arithmetic from $O(n^2)$ into $O(n)$. It reduces memory traffic and avoids quadratic HBM materialization for the attention matrix while computing exact dense attention.

22 FlashAttention generations: optimize the bottleneck you actually have

The durable concept is IO-aware exact attention. Later FlashAttention versions refine how that computation maps to newer GPU hardware.

FlashAttention-2 improved parallelism and work partitioning, including better distribution of work across thread blocks and warps and fewer non-matrix-multiply operations. [10]

FlashAttention-4, introduced in 2026 for Blackwell-class GPUs, addresses a different hardware balance: tensor-core throughput had grown faster than some surrounding resources, so bottlenecks shifted toward softmax/non-matmul work, shared-memory traffic, and pipeline structure. Its contribution is therefore not a new definition of attention but a hardware-aware algorithm/kernel co-design for a changed machine. [12]

This is an important engineering lesson:

Mental model. An algorithm can be mathematically unchanged while its fastest implementation changes as hardware changes. “Attention” is the mathematical operation; “FlashAttention-N” is a family of increasingly hardware-aware ways to execute that operation.

For an evergreen mental model, remember:

1. avoid unnecessary materialization,
2. tile data to exploit faster memory,
3. fuse compatible operations,
4. keep expensive compute units fed,
5. reconsider bottlenecks when hardware ratios change.

Benchmark numbers from individual FlashAttention generations are intentionally omitted here because they are hardware-, shape-, precision-, and software-version-dependent.

23 Sparse and local attention

Dense attention connects every query to every key, producing $O(n^2)$ pairwise interactions for same-length self-attention. Another family of approaches changes the *connectivity pattern itself*.

Longformer combines local windowed attention with task-motivated global attention and was designed so attention cost scales linearly with sequence length under its fixed-window pattern. [5]

BigBird combines sparse components such as local, global, and random connections, also reducing the full quadratic dependency while retaining important theoretical properties under its design. [6]

These approaches are conceptually different from FlashAttention:

Approach	What changes?	Exact full dense attention?
FlashAttention	execution / IO strategy	yes
Local/sparse attention	which query-key pairs are computed	no, connectivity is restricted/structured

Sparse attention can reduce arithmetic work by not computing all pairs. The trade-off is that the model no longer has unrestricted direct connectivity in a single layer; pattern design matters.

Important. “Efficient attention” is not one technique. Some methods preserve exact dense attention but execute it better; others change the attention pattern or approximation to reduce the number of interactions.

24 A transparent PyTorch implementation

The following code mirrors the mathematics for a simple single-/multi-head tensor layout. It is intentionally explicit rather than optimized.

```
import math
import torch

# q, k, v: (... , L/S, d_head)
scores = q @ k.transpose(-2, -1)
scores = scores / math.sqrt(q.size(-1))

if causal:
    L, S = q.size(-2), k.size(-2)
    # This simple mask assumes a square aligned case.
    mask = torch.triu(
        torch.ones(L, S, dtype=torch.bool, device=q.device),
        diagonal=1,
    )
    scores = scores.masked_fill(mask, float("-inf"))

weights = torch.softmax(scores, dim=-1)
out = weights @ v
```

The code exposes the exact conceptual stages:

$$QK^{\top} \rightarrow \text{scale} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow AV.$$

What it does *not* expose is how a production kernel fuses operations, handles numerically stable blockwise softmax, chooses hardware backends, represents causal bias in non-square cases, or supports grouped query heads.

Important. The simple triangular mask shown here is for an aligned square causal example. For real non-square cached decoding, use a library API whose causal alignment semantics are documented rather than assuming this mask is universally correct.

25 Production SDPA in PyTorch

Current PyTorch exposes scaled dot-product attention as

```
torch.nn.functional.scaled_dot_product_attention(  
    query, key, value,  
    attn_mask=None,  
    dropout_p=0.0,  
    is_causal=False,  
    scale=None,  
    enable_gqa=False,  
)
```

The documentation defines query length L , source/key length S , optional Boolean or additive masks, default scaling $1/\sqrt{d}$, causal bias behavior, dropout, and experimental grouped-query support. Depending on inputs and backend, PyTorch can dispatch to optimized implementations rather than literally executing the pedagogical Python sequence above. [17]

A few implementation lessons follow:

- API semantics matter: masks are not interchangeable across libraries. In current PyTorch SDPA, a Boolean `attn_mask` value of `True` means the position participates in attention; this is the inverse of `MultiheadAttention`'s Boolean `key_padding_mask` convention.
- Dropout behavior must be controlled explicitly when switching between training and evaluation logic.
- GQA has shape/divisibility constraints in current implementations.
- Kernel selection can depend on device, dtype, shape, masks, and feature support.

Implementation APIs evolve. The mathematics in this handbook is evergreen; production code should always be checked against the documentation for the installed framework version.

26 Common implementation mistakes

1. Softmax on the wrong dimension

Attention normalizes across keys for each query. Applying softmax across query rows changes the operation.

2. Forgetting the scale

Unscaled dot-product attention is not the Transformer formulation and can behave differently as head dimension changes.

3. Masking after softmax

Setting probabilities to zero after softmax without renormalizing is generally not equivalent to excluding those logits before normalization.

4. Using a square-mask assumption for $L \neq S$

Cached decoding and cross-attention often have different query/key lengths. Causal alignment must match the framework's convention.

5. Confusing head count with KV-head count

In GQA, query heads and key/value heads differ. Shape logic and cache formulas must use H_{kv} for K/V storage.

6. Treating attention weights as the layer output

The weights are an intermediate. The output is AV (followed in multi-head attention by concatenation/output projection and then the surrounding Transformer block operations).

7. Comparing “attention memory” without defining what is counted

Weights, temporary score matrices, KV cache, activations, and allocator overhead are different quantities.

Mental model. Most attention bugs are shape/axis/semantics bugs, not failures to remember the main formula. Track each dimension explicitly.

27 Common misconceptions

Misconception	Correction
“Q is a question; K is an answer.”	Q/K are learned vector projections whose dot product supplies compatibility. The analogy is not the mechanism.
“Attention is just similarity search.”	Dot products are learned task-specific compatibility scores; values are then aggregated.
“Attention weights are the output.”	They are coefficients; the output is a weighted sum of V.
“Self-attention means causal attention.”	Self-attention says Q/K/V come from the same stream. Causality is a mask/connectivity constraint.
“FlashAttention is linear attention.”	FlashAttention computes exact dense attention with an IO-aware execution strategy.
“KV caching makes attention constant time.”	It reuses past K/V, but a new dense query still interacts with a growing prefix.
“MQA/GQA reduce query heads.”	Their defining change is sharing/reducing KV heads relative to query heads.
“RoPE gives unlimited context.”	Position representation alone does not guarantee trained long-context competence.

A good technical explanation should remove these shortcuts early, because later systems concepts build directly on the distinctions.

28 Are attention weights explanations?

Attention weights are often visually compelling: they form distributions over input positions, so it is tempting to interpret high weight as “the model used this token because it mattered.” That conclusion is not generally justified without additional analysis.

Jain and Wallace tested attention weights as explanations across several NLP models and found cases where learned attention was weakly related to gradient-based importance measures and where very different attention distributions could yield similar predictions. They argued that standard attention weights should not automatically be treated as faithful explanations. [13]

Wiegrefe and Pinter challenged a categorical rejection, arguing that the answer depends on the definition of explanation and on appropriate baselines/tests. They proposed diagnostic conditions under which attention can still be informative. [14]

The safest technical conclusion is therefore narrower than either slogan:

Mental model. Attention weights are internal model quantities that can sometimes support analysis, but they are not automatically faithful causal explanations of a model’s prediction. Interpretability claims require a method and validation beyond merely plotting the weights.

This nuance matters especially in modern Transformers, where many heads/layers, residual pathways, feed-forward blocks, and later computations contribute to the final output.

29 The complete mental model

Attention is easiest to remember as a sequence of six ideas.

1. **Project.** Hidden states are mapped into learned query, key, and value representations.
2. **Compare.** Each query is compared with allowed keys, usually by a dot product.
3. **Scale.** Dot products are divided by $\sqrt{d_k}$ in scaled dot-product attention.
4. **Constrain.** Masks or biases define which interactions are allowed or favored.
5. **Normalize.** Row-wise softmax converts scores into weights over keys.
6. **Aggregate.** Those weights mix the corresponding values into the query's new representation.

Everything else in this handbook modifies one of the surrounding dimensions:

- **Multi-head attention** repeats the operation in multiple learned subspaces.
- **RoPE/position methods** make attention position-aware.
- **Causal masking** enforces autoregressive information flow.
- **KV caching** reuses past K/V during incremental decoding.
- **MQA/GQA** reduce KV-head multiplicity and cache/bandwidth cost.
- **MLA** compresses the cached KV representation with a different projection strategy.
- **FlashAttention** executes exact dense attention with less expensive memory traffic.
- **Sparse/local attention** changes which pairs are computed to reduce dense connectivity cost.

If these distinctions are clear, you do not merely know the attention equation. You have the conceptual foundation required to understand why modern LLM architectures and inference systems keep changing the structures around it.

References and further reading

References

- [1] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv:1409.0473; ICLR 2015.
- [2] Minh-Thang Luong, Hieu Pham, Christopher D. Manning. *Effective Approaches to Attention-based Neural Machine Translation*. EMNLP 2015. arXiv:1508.04025.
- [3] Ashish Vaswani et al. *Attention Is All You Need*. NeurIPS 2017. arXiv:1706.03762.
- [4] Noam Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv:1911.02150, 2019.
- [5] Iz Beltagy, Matthew E. Peters, Arman Cohan. *Longformer: The Long-Document Transformer*. arXiv:2004.05150, 2020.
- [6] Manzil Zaheer et al. *Big Bird: Transformers for Longer Sequences*. NeurIPS 2020. arXiv:2007.14062.
- [7] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, 2021.
- [8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Re. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. arXiv:2205.14135.
- [9] Joshua Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP 2023. arXiv:2305.13245.
- [10] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. arXiv:2307.08691, 2023.
- [11] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. arXiv:2405.04434, 2024.
- [12] Ted Zadouri et al. *FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling*. arXiv:2603.05451, 2026.
- [13] Sarthak Jain, Byron C. Wallace. *Attention is not Explanation*. NAACL-HLT 2019.
- [14] Sarah Wiegrefe, Yuval Pinter. *Attention is not not Explanation*. EMNLP-IJCNLP 2019.
- [15] Stanford CS224N. *Natural Language Processing with Deep Learning*, Winter 2026 materials; Assignment 3: Self-Attention and Transformers.
- [16] Stanford CS336. *Language Modeling from Scratch*, current course materials; systems assignment includes a Triton FlashAttention-2 implementation.
- [17] PyTorch documentation. `torch.nn.functional.scaled_dot_product_attention`. Current documentation consulted for API shapes, masks, causality, scaling, dropout, and GQA behavior.

Research note: equations, tensor-shape statements, the worked numerical example, and implementation-sensitive claims were independently checked during manuscript preparation. Primary papers are used for named methods; current framework behavior should always be rechecked against the installed version.